

Practical Comparisons and Common Failure Cases

Frequently confused concepts and realistic failure patterns across the stack.

Engineering notes and worked references

Contents

1. Embeddings vs vector database
2. Vector search vs SQL
3. RAG vs conversation memory
4. Query rewriting vs answering
5. Tool use vs normal generation
6. Retrieval failure
7. Answer-generation failure
8. Session leakage
9. Invalid structured output
10. Noisy top-k
11. Stale summary memory

1. Embeddings vs vector database

Embeddings are numeric representations created by a model. A vector database or index stores those vectors and searches them efficiently.

The model and the store solve different problems. You can create embeddings without Chroma, and you can place vectors from many embedding models into vector systems as long as the dimensions and index are managed correctly.

Example from use

all-MiniLM-L6-v2 creates a 384-number vector. Chroma can store it with metadata.

FAISS searches vectors but does not generate their meaning.

Things people mix up

- Calling Chroma the embedding model.
- Thinking an embedding is a database record by itself.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in embeddings vs vector database is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

2. Vector search vs SQL

Vector search is useful for semantic similarity in text. SQL is useful for exact structured filters, joins, counts, and stored application data.

A knowledge assistant may use vector search for note content and SQLite for sessions or saved notes. These are complementary, not competing, tools.

Example from use

Find passages similar to 'how does top-k work?' with vectors.

Count saved notes per session with SQL.

Things people mix up

- Embedding rows just to count them.
- Using SQL LIKE as the only semantic search.
- Using vector similarity to enforce session ownership.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in vector search vs sql is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

3. RAG vs conversation memory

RAG brings external source material into the current request. Conversation memory preserves earlier interaction context.

A follow-up may need both: memory to understand what 'that' refers to, then RAG to retrieve the source that answers the resolved question.

Example from use

History says the user was discussing top-k. Rewrite the follow-up, then retrieve the top-k section.

Memory alone should not replace the source notes when the answer must be grounded.

Things people mix up

- Treating chat history as a vector knowledge base.
- Sending RAG chunks as if they were previous assistant messages.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?

- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in rag vs conversation memory is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

4. Query rewriting vs answering

Query rewriting creates a better search query. It is not the final user answer.

The rewritten query should be concise and faithful. Then the retrieval and answer steps can use it.

Example from use

User: 'what about top-k?' Rewrite: 'How does top-k affect vector retrieval?'.

The final answer can then explain the tradeoff in normal language.

Things people mix up

- Returning the rewritten query as the answer.
- Adding facts during rewrite.
- Rewriting a clear new topic into the old topic.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in query rewriting vs answering is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

5. Tool use vs normal generation

Tools are for actions or exact lookups the model cannot perform by itself, such as saving a note or reading a database.

Normal explanation text does not need a tool unless the answer depends on stored or external state.

Example from use

'Save this explanation' uses `save_note`.

'Thanks' needs no tool.

Things people mix up

- Calling tools to make the system look advanced.
- Using the model to pretend a save happened.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in tool use vs normal generation is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

6. Retrieval failure

Retrieval failure means the needed note chunk is not in the context sent to the model.

The first debugging step is to inspect the rewritten query, retrieved chunk IDs, scores, and filters.

Example from use

Correct source exists but top-k results are unrelated.

A metadata filter accidentally restricts search to FastAPI notes.

Things people mix up

- Changing the final answer prompt first.
- Blaming hallucination when the source never arrived.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in retrieval failure is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

7. Answer-generation failure

Answer-generation failure means the correct source is present but the answer ignores it, misreads it, or adds unsupported detail.

Check grounding instructions, ordering, output schema, and whether conflicting context was also included.

Example from use

Top result clearly says 'stop value is excluded' but answer says included.

Source supports 12 months but answer invents unlimited.

Things people mix up

- Re-embedding the corpus when retrieval is correct.
- Increasing top-k and adding more noise.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in answer-generation failure is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

8. Session leakage

Session leakage happens when one user's messages or saved state appear in another session.

The cause is usually application storage or filtering, not the model having mysterious memory.

Example from use

A global history list is shared by every request.

A SQL query forgets `WHERE session_id = ?`.

Things people mix up

- Trying to fix leakage with a prompt.
- Assuming the provider remembered the other user.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in session leakage is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

9. Invalid structured output

The model can return text that cannot be parsed or an object that misses required fields.

Treat parsing and schema validation as normal error points and decide whether to retry, repair, or return an error.

Example from use

`json.loads` fails because prose appears before the JSON object.

The JSON parses but `quiz` is a string instead of a list.

Things people mix up

- Catching the exception and pretending the result is empty but valid.
- Using regex to patch every malformed response.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in invalid structured output is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?

- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

10. Noisy top-k

A high top-k can fill the prompt with loosely related chunks, repeated overlap, or adjacent topics.

More evidence is useful only when it actually supports the question. Inspect the ranking and remove unnecessary context.

Example from use

Top-15 returns memory, Git, and API notes for a narrow embedding question.

Top-3 returns the exact definition and two comparison passages.

Things people mix up

- Assuming more context always improves reasoning.
- Counting duplicated overlapping chunks as separate support.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in noisy top-k is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.

11. Stale summary memory

A summary can become stale when the user changes a choice or topic but the application keeps reusing the old summary.

Structured state and recent exact messages can prevent an old summary from overriding the latest request.

Example from use

Summary says provider=Groq, later user switches to OpenRouter, but stale summary is still sent.

Update or regenerate summary after important state changes.

Things people mix up

- Treating summary text as higher priority than explicit new user input.
- Never checking when the summary was created.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How to decide which side applies

The comparison in stale summary memory is useful because the two ideas often appear in the same application but solve different parts of the problem. When choosing between them, ask what data is available, whether the operation must be exact, and whether the result needs to change stored state.

If a bug seems to involve both sides, inspect their boundary. For example, session memory can resolve a vague follow-up, but vector retrieval still has to find the supporting note. A good rewrite does not guarantee a good search, and a good search does not guarantee a grounded answer.

Diagnostic questions

- Which component owns the data involved?
- Which step can be checked deterministically?
- What intermediate output would prove the step worked?
- Would another component be more appropriate for exact structured data?

Keeping these boundaries clear prevents one technology from being stretched into jobs it was not chosen to do.